

Facultade de Informática
UNIVERSIDADE DA CORUÑA

TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN COMPUTACIÓN



Lifelong Adaptation in Minimal Structures: A Spiking-NEAT Architecture with R-STDP

Estudante: Naiara Manteiga Cendón

Dirección: Outro Nome Completo

A Coruña, October 2026.

Dedicatoria

I would like to thank …

Acknowledgements

I would like to thank …

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Resumo

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Keywords:

- First item in a list
- Second item in a list
- Third item in a list
- Fourth item in a list
- Fifth item in a list
- Sixth item in a list
- Seventh item in a list

Palabras chave:

- First item in a list
- Second item in a list
- Third item in a list
- Fourth item in a list
- Fifth item in a list
- Sixth item in a list
- Seventh item in a list

Contents

1 Introduction **1**

1.1 Motivation . . . 1

1.2 Objectives . . . 2

1.2.1 Main objective . . . 2

1.2.2 Specific objectives . . . 3

1.3 Chosen approach . . . 3

2 Theoretical Background **6**

2.1 Neuroevolution and Genetic Algorithms . . . 6

2.1.1 TWEANNs . . . 7

2.1.2 NeuroEvolution of Augmenting Topologies (NEAT) . . . 8

2.1.2.1 Innovation numbers . . . 9

2.1.2.2 Speciation . . . 10

2.1.2.3 Minimal structures . . . 10

2.1.2.4 Why NEAT? . . . 11

2.1.2.5 Limitations . . . 11

2.2 Green AI and Spiking Neural Networks (SNNs) . . . 11

2.2.1 Green AI . . . 11

2.2.2 SNN fundamentals and the LIF model . . . 12

2.2.3 Training spiking networks . . . 13

2.2.4 Theoretical energy estimation . . . 14

2.3 Synaptic Plasticity and Reward-Modulated Learning . . . 15

2.3.1 Spike-Timing-Dependent Plasticity (STDP) . . . 15

2.3.2 Reward-Modulated STDP (R-STDP) . . . 15

3 State of the Art **17**

3.1 Game-playing controllers . . . 17

3.2 Spiking networks for control . . . 18

3.3 Reward-modulated plasticity in spiking agents 19
3.4 Evolved plastic networks 19
3.5 Energy-aware evaluation and neuromorphic hardware 20
3.6 Comparative analysis and research gap 20

Bibliography **24**

List of Figures

2.1 Architecture of a feedforward neural network illustrating the forward pass of activations and the backward propagation of error adjustments for weight optimization. Adapted from GeeksforGeeks. 6
2.2 The competing conventions problem. 8
2.3 Matching genomes with innovation numbers. 10

List of Tables

3.1 Positioning of related work. “n/r”: not reported in the cited work. “Local plasticity” means that weights change through local (STDP-type) rules; it does not indicate whether learning continues after deployment. 22

Chapter 1
Introduction

This project can be found on GitHub at the following link: https://github.com/nmanteiga/TFG_Spiking-NEAT_Arquitecture_with_R-STDP

1.1 Motivation

Artificial Neural Networks (ANNs) have been at the forefront of computer science research for the past few decades, and the idea of building artificial systems inspired by the way the brain processes information remains a prominent research focus. In recent years, deep learning and transformer architectures have advanced substantially, driven by their remarkable results and by the commercialization of General Purpose Artificial Intelligence (GPAI). Nonetheless, these models suffer from two major shortcomings: their high energy and environmental cost, and the large amounts of data or environment interactions they require for training. This energy cost has given rise to initiatives such as Green AI [5], which advocates for computational efficiency as an evaluation criterion alongside accuracy. Neuroevolutionary algorithms offer a gradient-free alternative: they optimise neural networks with evolutionary search instead of backpropagation. In particular, the NeuroEvolution of Augmenting Topologies (NEAT) algorithm [1] improves on earlier methods by evolving network structure together with weights, and it was shown to solve benchmark control problems such as double pole balancing more efficiently than earlier neuroevolution methods, as detailed in Chapter 2.

The practical application of this project is inspired by a well-known demonstration of NEAT playing Super Mario World, an implementation its creator named MarI/O [6]. It is an informal demonstration rather than a controlled study, but it illustrates that NEAT needs neither a labelled training dataset nor a model of the environment: MarI/O starts without knowing the controls, discovers movement by trial and error, and evolves until it can clear the level. This addresses the first shortcoming, the dependence on large training datasets. This project uses Super Mario Bros instead of Super Mario World because it is available as

a Python environment (Section 1.2.1) and is less hardware dependent, so that the measured behaviour is closer to what a resource-limited robot controller would show.

However, standard NEAT does not address the energy problem on its own. To address it, this project combines NEAT with two biologically inspired principles: Spiking Neural Networks (SNNs) [2] and Reward-Modulated Spike-Timing-Dependent Plasticity (R-STDP) [4, 8]. SNNs are built on the premise that biological neurons communicate through discrete electrical pulses called spikes. As an analogy, a standard ANN behaves like an open tap, through which water (signal) flows continuously; an SNN behaves like a tap that is almost closed, in which water accumulates and falls only as a single drop once a certain level is reached. R-STDP is intended to complement the evolutionary process: while NEAT evolves the topology and the initial weights across generations, R-STDP adapts the synaptic weights during gameplay. This combination is expected to give the agent more adaptability within an episode; whether it also helps the agent to transfer to new levels or similar tasks without restarting training from scratch is a question that this project does not claim to answer in advance.

The practical relevance of this architecture extends to constrained engineering environments. A rover or search-and-rescue drone could use local plasticity to adapt after a component failure without relying on a remote server. Remote Internet-of-Things (IoT) sensors could remain mostly idle and compute only when an acoustic or environmental anomaly is detected. Medical devices such as neural implants or smart pacemakers could process changing biological signals locally, while wearable devices could personalise their models without continuously transmitting private data to the cloud. This project evaluates the approach only in a simulated game, but these scenarios illustrate the potential uses of such a technology.

As modern technology increasingly shifts toward edge computing, autonomous robotics and IoT devices, the demand for AI systems that adapt in real time with low power consumption is growing. Evaluating spiking networks with R-STDP in a dynamic software environment such as Super Mario Bros therefore addresses a gap in the literature (Chapter 3) and contributes a first step towards controllers that could later be deployed on neuromorphic hardware.

1.2 Objectives

1.2.1 Main objective

The primary objective of this project is to develop an autonomous program, commonly referred to as an “agent”, capable of navigating and completing levels of Super Mario Bros. In this context, an agent is a computational entity that repeatedly observes its surroundings and makes decisions to achieve a goal. The agent learns from a reward signal, that is, through trial, error and feedback from the environment, as in Reinforcement Learning (RL). In this

project the reward guides two mechanisms: the evolutionary search (through the fitness of each genome) and the local synaptic adaptation within an episode (through R-STDP).

To address the energy cost of conventional networks, the project replaces the continuous-valued neurons used by standard NEAT with a biologically inspired Spiking Neural Network (SNN) built from Leaky Integrate-and-Fire neurons. The agent's within-episode learning is governed by Reward-Modulated Spike-Timing-Dependent Plasticity (R-STDP), which lets it adapt its synaptic weights while it plays.

The goal is to evaluate whether an agent built on these principles can adapt during play while reducing the estimated energy consumption relative to a standard NEAT baseline.

The experimental environment is the Super Mario Bros implementation provided by the gym-super-mario-bros package in Python [12].

For this document, *comparable performance* is defined operationally: the spiking agent should reach at least 90% of the baseline mean distance and completion rate, while requiring no more than 10% additional frames to complete the level. These metrics will be reported as mean and standard deviation over five randomised seeds on Level 1-1. Training energy is outside the primary energy metric; the total number of evaluated frames will nevertheless be reported for transparency.

1.2.2 Specific objectives

To achieve the main objective, the project is divided into the following sub-goals:

- **Establish a baseline:** implement and train a standard NEAT algorithm to play Super Mario Bros, to serve as a performance and architectural baseline.
- **Implement spiking mechanics:** replace the continuous activation functions of standard NEAT (e.g., sigmoid) with a discrete-time Leaky Integrate-and-Fire (LIF) [2] neuron model, and define a spike-encoding method for the environment observation.
- **Integrate local plasticity:** incorporate R-STDP so that the agent's synaptic weights adapt during an episode according to the environmental reward.
- **Evaluate efficiency:** compare the performance and the estimated inference energy of the Spiking-NEAT agent with those of the standard NEAT baseline.

1.3 Chosen approach

The approach is organised around three components, described here through the metaphor of the architect, the engine and the teacher.

The architect (topology). The structure of the agent's network is generated with NEAT. NEAT improves upon earlier Topology and Weight Evolving Artificial Neural Networks (TWEANNs) through historical markings (innovation numbers), speciation and incremental growth from minimal structures. Because NEAT has been validated on control benchmarks such as double pole balancing [1], and because it was the algorithm used in the MarI/O demonstration, it is a natural choice to grow the network topology in this project.

The engine (neuron model). Standard NEAT typically uses a continuous sigmoidal activation function bounded in $[0, 1]$. Because activations are continuous floating-point values, the hardware must perform a Multiply-Accumulate (MAC) operation for every synapse at every time step, even when a neuron's activation is close to zero. This unconditional computation is what drives the energy cost discussed in the previous section. To reduce it, the network is turned into an SNN by using the Leaky Integrate-and-Fire (LIF) model [2]. The LIF model is a mathematical abstraction of a biological neuron with three mechanics: *integrate* (accumulate incoming signals), *leak* (the membrane potential decays over time) and *fire* (emit a binary spike when the potential reaches a threshold, then reset). Extending the tap analogy, the neuron is a bucket with a small hole: the tap is the synaptic current (integrate), the water level is the membrane potential, the hole is the leak, and the bucket overflowing and being emptied is the spike (fire). Since the signals between neurons are binary, the contribution of a spike to the next neuron is a single addition instead of a multiplication, and nothing is transmitted when a neuron does not fire. If activity is sparse, this reduces the number of expensive operations. The membrane potential of every neuron must still be updated at each time step, and this cost is included explicitly in the energy model of Section 2.2.4.

The teacher (learning rule). In standard NEAT, synaptic weights change only through random mutation between generations, which can be an inefficient way to refine them. The project therefore adds R-STDP as the "teacher". R-STDP is a biologically inspired, reward-driven rule: it keeps a temporary trace of which synapses were recently active just before a postsynaptic spike, and when a reward signal arrives, it strengthens or weakens those synapses depending on whether the reward was better or worse than expected. It is inspired by the role of dopamine in the brain, and it favours connections that preceded rewarded behaviour.

NEAT evolves the topology and the initial weights, while R-STDP adapts the effective weights during each episode. The adapted weights are reset to the genome-encoded initial values at the beginning of each new episode and are not written back to the genome. Several episodes per generation will therefore be evaluated so that the effect of within-episode adaptation can influence fitness. As an ablation, a Lamarckian variant will also be evaluated, in which the weights adapted by R-STDP at the end of an episode are written back to the genome so that offspring inherit them (as opposed to the main design, where the adaptation

is discarded after each episode).

Chapter 2

Theoretical Background

This chapter introduces the theoretical and mathematical foundations needed to follow the rest of this document. It describes the models and algorithms behind neuroevolution, Spiking Neural Networks (SNNs) and biologically inspired learning rules, and it explains why the three are combined in the proposed Spiking-NEAT architecture for dynamic control tasks. Related work is reviewed separately in Chapter 3.

2.1 Neuroevolution and Genetic Algorithms

The standard method for training Artificial Neural Networks (ANNs) is the backpropagation algorithm. Its purpose is to optimise a network's predictions by systematically adjusting its weights and biases to minimise a measure of error. In its classical, supervised form it learns from examples for which the correct output is known.

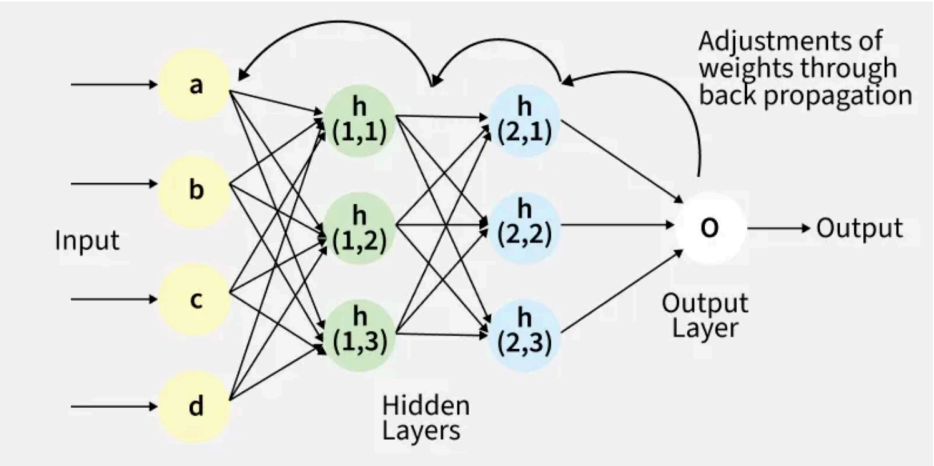


Figure 2.1: Architecture of a feedforward neural network illustrating the forward pass of activations and the backward propagation of error adjustments for weight optimization. Adapted from GeeksforGeeks.

As illustrated in Figure 2.1, training operates in four phases:

1. **Forward pass:** the input propagates forward through the network's layers. Each node applies a weighted sum and a nonlinear function until the final layer outputs a prediction.
2. **Error calculation:** a loss function measures the discrepancy (the error) between the prediction and the "ground truth" provided by the training dataset.
3. **Backward pass:** the algorithm computes the gradient of the loss and moves backward through the network, assigning to each weight and bias its share of the error.
4. **Optimizer step:** an optimizer uses these gradients to update the weights so that the next prediction improves.

Gradient descent is very powerful for supervised tasks such as image classification, but it has two requirements: it needs a large set of labelled examples to compute the loss, and it requires differentiable activation functions to compute the gradients.

In dynamic control problems, such as autonomous robotics or video games like Super Mario Bros, there is no dataset of "perfect moves". The agent only receives a sparse reward that tells it, after the fact, how well it did. Backpropagation can still be applied in this setting, as deep reinforcement learning shows [13], but it then needs a very large number of environment interactions and still requires differentiable networks. Neuroevolution (NE) is an alternative that avoids gradients altogether.

Neuroevolution applies the principles of natural selection to neural networks. Instead of training a single network, it uses a Genetic Algorithm (GA) to maintain a "population" of candidate networks. Each network is evaluated in the environment and receives a score called its *fitness*. The weakest networks are discarded, while the best ones are selected to produce the next generation through *crossover* (combining the genes of two parents) and *mutation* (random changes to weights or structure). Since fitness is computed only from the outcome of an episode, neither gradients nor labelled data are needed.

2.1.1 TWEANNs

Early neuroevolution systems optimised only the synaptic weights of a network whose topology was fixed by a human designer. Choosing a good architecture for a complex task is difficult, which motivated Topology and Weight Evolving Artificial Neural Networks (TWEANNs): algorithms that evolve both the connection weights and the structure of the network. Stanley and Miikkulainen [1] identify three problems that limited earlier TWEANNs, and that NEAT was designed to solve.

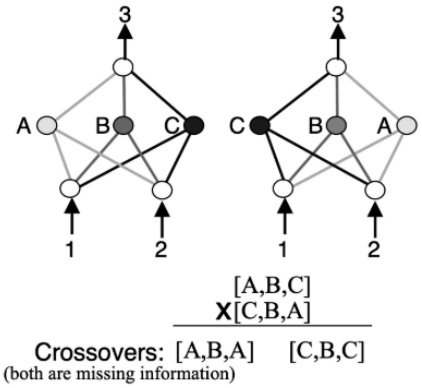


Figure 2.2: The competing conventions problem. Two networks that compute the same function but list their hidden units in a different order are incompatible for crossover: both single-point recombinations lose one of the three components of the solution. Reproduced from Figure 1 of [1].

1. **Competing conventions (permutation) problem.** Two parent networks may solve the same problem while encoding their solutions with their hidden nodes in a different order. When such parents are crossed over (Figure 2.2), the offspring often lose or duplicate parts of the solution, because the algorithm has no way to align the mismatched hidden nodes. Crossing two good parents can therefore produce a damaged offspring.

2. **Unprotected innovation.** A newly added structure, such as a hidden node, usually lowers fitness at first, until its weights are tuned. In a single unified population, such innovations are quickly eliminated by older and simpler networks whose weights are already tuned. A single pool also lets the initially fittest individuals take over the population, which reduces diversity.

3. **Topological bloat.** To ensure diversity, early methods started from populations of random, complex topologies. This filled the genomes with unjustified connections from the first generation and enlarged the search space unnecessarily.

As Figure 2.2 shows, crossing over two functionally equivalent networks with different internal orderings produces offspring that miss information (for example, node C or node A).

These three problems made it hard for TWEANNs to scale to larger tasks. NEAT addresses each of them with one mechanism.

2.1.2 NeuroEvolution of Augmenting Topologies (NEAT)

NeuroEvolution of Augmenting Topologies (NEAT) [1] addresses the limitations above through three mechanisms: crossover based on historical markings, protection of structural innova-

tion through speciation, and incremental growth from minimal structures. Together, they allow evolution to optimise and complexify solutions at the same time.

2.1.2.1 Innovation numbers

The mechanism that resolves the competing conventions problem is the use of historical markings, or innovation numbers. In biology, homology allows organisms with different genetic structures to recombine because their genes share a common origin. NEAT uses an analogous idea: every new structural mutation (a new connection or a new hidden node) that appears in the population is assigned a unique, increasing integer, its global innovation number. Two genes with the same innovation number therefore descend from the same original mutation.

This marking turns the problem of comparing different topologies into a linear alignment of gene lists. When two parents are crossed over, their genomes are aligned by innovation number (Figure 2.3) and the genes are classified into three types:

- **Matching genes:** genes with the same innovation number in both parents. The offspring inherits each of them from a randomly chosen parent.
- **Disjoint genes:** genes present in only one parent, whose innovation number lies within the range of the other parent's innovation numbers.
- **Excess genes:** genes present in only one parent, whose innovation number lies beyond the range of the other parent's innovation numbers (at the end of the genome).

Disjoint and excess genes are inherited from the fitter parent; if both parents have equal fitness, they are inherited randomly. In addition, there is a preset probability that a gene remains disabled in the offspring if it is disabled in either parent. With this alignment, networks of very different sizes and topologies can be recombined without losing track of which gene performs which function.

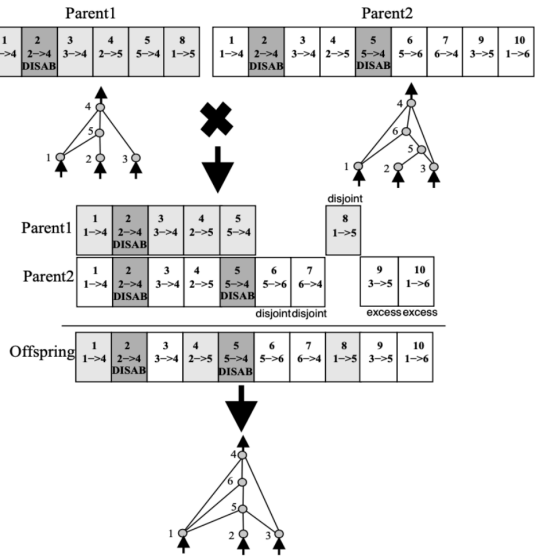


Figure 2.3: Crossover with innovation numbers. The numbers at the top of each gene show which genes of the two parents match, so their genomes can be aligned without analysing the topology. Matching genes are inherited randomly; disjoint and excess genes come from the fitter parent (here equal fitness is assumed, so they are inherited randomly too). Reproduced from Figure 4 of [1].

2.1.2.2 Speciation

The second mechanism is speciation, also known as niching. When a structural mutation is added, performance often drops temporarily until the new weights are optimised. If the whole population competes in a single pool, these new topologies are driven to extinction by older, simpler networks whose weights are already tuned.

To prevent this, NEAT divides the population into species according to topological similarity, which it measures by using the innovation numbers described above: genomes with many matching genes are placed in the same species, whereas genomes with many disjoint or excess genes are placed in different ones.

Within each species, NEAT applies explicit fitness sharing: individuals in the same niche share their fitness, so that a large species cannot take over the whole population. Consequently, new structures compete mainly against similar genomes rather than against the whole population, which gives them the time to optimise their weights before they are judged.

2.1.2.3 Minimal structures

The third mechanism concerns how evolution starts. NEAT begins with a population of minimal networks that have no hidden nodes: the inputs are connected directly to the outputs.

Structure is added gradually, through mutations that insert new connections and new hidden nodes. A new structure persists only if it improves fitness enough to survive selection.

Growing from minimal structures keeps the search space small at the beginning and tends to produce compact networks, because added complexity must justify itself. It also shows that evolution can optimise and complexify solutions at the same time, in analogy to biological evolution.

2.1.2.4 Why NEAT?

Gradient-based deep reinforcement learning can play games such as Super Mario Bros, but it needs very large numbers of environment interactions [13]. Neuroevolution needs no gradients and no labelled dataset, only a fitness score per episode. Stanley and Miikkulainen [1] report, through ablation studies and benchmark comparisons on the XOR problem and on double pole balancing, that NEAT was more efficient than the earlier neuroevolution methods they compared against. They attribute this to the three mechanisms above, which together prevent the three problems of TWEANNs.

For these reasons, and because it is the algorithm used in the MarI/O demonstration [6], NEAT is chosen in this project to grow the network topology and as the baseline for the subsequent integration of spiking neurons and R-STDP.

2.1.2.5 Limitations

Standard NEAT networks rely on continuous activation functions, such as the sigmoid function, whose outputs lie in $[0, 1]$. Because activations are continuous floating-point values, the hardware must perform a Multiply-Accumulate (MAC) operation for every synapse at every time step, even when a node's activation is close to zero.

This unconditional computation makes inference energy-intensive, which is a problem for edge-computing scenarios such as autonomous robotics, aerospace applications and Internet-of-Things (IoT) devices, where energy budgets are tight. This motivates replacing the continuous neurons with spiking ones.

2.2 Green AI and Spiking Neural Networks (SNNs)

2.2.1 Green AI

The rapid rise and commercialization of AI and GPAI has been accompanied by growing concern about the computational cost of training and running these systems. Schwartz et al. distinguish two research cultures. "Red AI refers to AI research that seeks to obtain state-of-the-art results in accuracy (or related measures) through the use of massive computational

power—essentially 'buying' stronger results" [5]. Green AI, in contrast, advocates for treating computational cost and energy efficiency as evaluation criteria alongside accuracy [5]. This project follows the Green AI view and reports energy as a core metric next to performance.

Schwartz et al. describe the cost of producing an AI result R as proportional to the cost of executing the model on one example, the size of the dataset, and the number of hyperparameter experiments:

$$\text{Cost}(R) \propto E \cdot D \cdot H, \tag{2.1}$$

where E is the execution cost for one example, D is the dataset size, and H is the number of experiments.

Conventional networks are expensive to execute in part because they use continuous floating-point weights and activations, so the hardware performs a MAC operation for every active synapse at every time step. The activation function is the nonlinear function that maps a node's weighted input to its output; widely used examples are the sigmoid and the ReLU, both of which produce continuous values.

The biological brain does not operate like an open tap: neurons exchange discrete pulses. A biological neuron has three main parts: the dendrites, which receive input; the soma (cell body), which integrates it; and the axon, which transmits the output [2]. When the accumulated voltage at the soma is sufficiently high, the neuron emits an action potential, a brief pulse called a spike. A sequence of spikes over time is a spike train. These mechanisms are the basis of SNNs [2]. Because spikes are binary events, a spiking network can in principle replace multiplications by additions and does no transmission when there is no spike. This can be energy-efficient if activity is sparse and the hardware is designed to exploit it, as in neuromorphic processors.

2.2.2 SNN fundamentals and the LIF model

An SNN needs a mathematical model of the neuron. In this project it is the Leaky Integrate-and-Fire (LIF) model [2]. As its name indicates, it has three stages: integrate, leak and fire. First, the neuron accumulates the input from incoming spikes, which raises its membrane potential (integration). Second, the membrane is not a perfect integrator: if input spikes do not arrive often enough, the potential decays back toward its resting value (leak). Finally, when the potential reaches a threshold, the neuron emits a binary spike and its potential is reset to a baseline value (fire). In the proposed architecture, each continuous node generated by NEAT is replaced by an LIF neuron.

The LIF model can be written as an electrical circuit with membrane capacitance C, membrane resistance R, input current $I(t)$ and membrane potential $u(t)$ [2]:

$$I(t) = I_R(t) + I_C(t), \qquad I_R(t) = \frac{u(t) - u_{\text{rest}}}{R}, \qquad I_C(t) = C\frac{du(t)}{dt}. \tag{2.2}$$

Combining these terms gives the membrane equation

$$\tau_m \frac{du(t)}{dt} = -\left(u(t) - u_{\text{rest}}\right) + RI(t), \qquad \tau_m = RC. \tag{2.3}$$

When $u(t)$ reaches the threshold ϑ, the neuron emits a spike and its potential is reset to u_{reset}.

Simulations on a digital computer use a discrete-time version of this model, in which the potential is updated once per time step. In the form used in this project, the potential first accumulates the incoming current I_t, is then multiplied by a leak factor $\beta \in (0, 1)$ (the discrete counterpart of the exponential decay with time constant τ_m), and the neuron spikes if the result reaches the threshold:

$$u_t = \beta \, \max\!\big(u_{t-1} + I_t,\; u_{\text{rest}}\big), \qquad s_t = \begin{cases} 1, & u_t \geq \vartheta, \\ 0, & \text{otherwise}, \end{cases} \tag{2.4}$$

with u_t reset to u_{reset} after a spike. The max keeps the potential from falling below the resting value, which is taken as zero (and equal to the reset value) in the implementation.

Combining NEAT with LIF neurons gives the proposed Spiking-NEAT architecture. This has a consequence for training: spikes are binary events and therefore not differentiable, so such networks cannot be trained directly with standard backpropagation, and gradient-based approaches need workarounds such as surrogate gradients. A Spiking-NEAT architecture therefore requires a learning rule that does not depend on gradients. Reward-modulated plasticity, introduced in Section 2.3.2, is one such rule.

2.2.3 Training spiking networks

SNNs are difficult to train with standard backpropagation because of the non-differentiable nature of spikes. Surrogate-gradient methods [14] address this by replacing the derivative of the spike function with a smooth approximation during the backward pass, which makes gradient-based optimisation possible; however, backpropagation through time requires storing the network's state over many time steps, which makes it costly for continuous, online learning. For an autonomous agent that must adapt while it acts, local learning rules, such as Spike-Timing-Dependent Plasticity (STDP) and its reward-modulated variant, are an attractive alternative, because each synapse is updated using only information available locally.

Integrating SNNs with neuroevolution is another way to obtain effective networks without gradients. For instance, Qiu et al. [3] applied NEAT to evolve recurrent spiking controllers for nonlinear control problems in continuous domains. In their work, evolution determined

the weights and topology of the spiking network, which then stayed fixed during operation. Combining evolved topologies with local plasticity for adaptation within an episode has been explored only on small tasks (Section 3.4).

2.2.4 Theoretical energy estimation

The energy comparison in this project is limited to arithmetic operations at inference time. A conventional controller is modelled by its Multiply-Accumulate (MAC) operations and its activation-function evaluations, while the spiking controller is modelled by its Synaptic Operations (SynOps), its membrane updates and the input encoding:

$$E_{\text{ANN}} = N_{\text{MAC}} E_{\text{MAC}} + N_{\text{act}} E_{\text{act}}, \tag{2.5}$$

$$E_{\text{SNN}} = N_{\text{SynOp}} E_{\text{AC}} + N_{\text{update}} E_{\text{update}} + E_{\text{encoding}}. \tag{2.6}$$

Here N_{MAC} is the number of MAC operations, N_{act} the number of activation-function evaluations, N_{SynOp} the number of synaptic operations (one accumulate per spike delivered through a synapse), and N_{update} the number of membrane updates, each with its corresponding energy cost per operation.

Using the commonly cited 45-nm estimates of Horowitz [15], a 32-bit floating-point MAC costs approximately $4.6\,\text{pJ}$ and an accumulate (addition) costs approximately $0.9\,\text{pJ}$. The reported efficiency metric is the ratio $E_{\text{SNN}}/E_{\text{ANN}}$, together with the operation counts used to obtain it.

In the SNN, N_{update} is the total number of membrane updates, in which the potential is multiplied by the leak factor at every time step. Because this requires a multiplication, E_{update} is conservatively treated as equal to a MAC ($4.6\,\text{pJ}$), which avoids underestimating the SNN cost. Because the SNN processes spikes instead of raw pixel values, the operations needed to convert the observation grid into spike trains over the internal time steps are counted separately as E_{encoding}, summed over the evaluation episode.

The model covers arithmetic only and ignores memory-access costs, even though moving data between memory and the processor typically consumes more energy than arithmetic. The ratio $E_{\text{SNN}}/E_{\text{ANN}}$ should therefore be read as a comparative estimate under explicit assumptions, not as a prediction of power consumption on a specific device.

2.3 Synaptic Plasticity and Reward-Modulated Learning

2.3.1 Spike-Timing-Dependent Plasticity (STDP)

The conventional architectures discussed so far rely on continuous activation functions and are trained by backpropagation, which computes gradients of a loss function and therefore requires continuous values. The Spiking-NEAT approach uses binary spikes, so gradient descent cannot be applied directly. Relying only on NEAT's random weight mutations between generations is a slow way to tune weights. This project therefore proposes a biologically inspired alternative that adapts the synaptic weights while the agent interacts with the environment.

In biological brains, learning is associated with synaptic plasticity, the ability of connections between neurons to strengthen or weaken depending on their activity. This idea goes back to the Hebbian postulate: if neuron A repeatedly and persistently takes part in firing neuron B, the efficacy of the synapse from A to B increases [2, 7]. It is often summarised as "neurons that fire together wire together". Spike-Timing-Dependent Plasticity (STDP) is a formalisation in which the weight change depends on the precise timing of spikes. If a presynaptic neuron fires shortly before a postsynaptic neuron, the connection is strengthened (Long-Term Potentiation, LTP); if it fires shortly after, the connection is weakened (Long-Term Depression, LTD).

For a spike-time difference $s = t_{\text{post}} - t_{\text{pre}}$, a standard pair-based STDP learning window is

$$W(s) = \begin{cases} A_+ \exp(-s/\tau_+), & s > 0, \\ -A_- \exp(s/\tau_-), & s < 0, \end{cases} \tag{2.7}$$

where A_+ and A_- control the amount of potentiation and depression, while τ_+ and τ_- determine their temporal ranges. In R-STDP (next section), this window defines the pre-post contribution $F_{ij}(t)$ that feeds the eligibility trace.

Standard STDP is unsupervised: it considers only the relative timing of spikes and has no information about whether the resulting behaviour was good or bad for the agent. Consequently, if a harmful action is repeated, STDP can keep strengthening the pathways that produce it simply because the neurons fire in sequence. Without a goal-related signal, STDP alone can therefore reinforce unhelpful behaviour.

2.3.2 Reward-Modulated STDP (R-STDP)

Reward-modulated STDP (R-STDP) addresses this limitation by adding a reward signal [4, 8, 9, 10, 11]. It addresses the *distal reward problem*: a reward usually arrives some time after the spikes that caused it, so the network must remember which synapses were recently ac-

tive. R-STDP expands standard STDP into a three-factor learning rule, whose factors are the presynaptic activity, the postsynaptic activity and a global neuromodulatory signal that plays a role similar to dopamine in biological brains. When a presynaptic and a postsynaptic spike occur in sequence, STDP does not change the weight immediately: it updates a temporary *eligibility trace* on that synapse. The weight changes only when the neuromodulatory signal arrives, in proportion to the trace. If the agent receives more reward than expected, the traced synapses are strengthened; if it receives less than expected, they are weakened [4]. In this way, the learning is guided by the outcome of the agent's behaviour and not only by spike timing.

The eligibility trace and the reward-modulated weight update can be expressed as

$$\frac{de_{ij}(t)}{dt} = -\frac{e_{ij}(t)}{\tau_e} + F_{ij}(t), \qquad \frac{dw_{ij}(t)}{dt} = d(t)e_{ij}(t), \tag{2.8}$$

where $F_{ij}(t)$ is the instantaneous pre-post STDP contribution of the synapse from neuron j to neuron i, τ_e is the trace time constant, and

$$d(t) = R(t) - \langle R \rangle \tag{2.9}$$

is the neuromodulatory reward-prediction error, that is, the difference between the current reward $R(t)$ and its running average $\langle R \rangle$. Positive values strengthen tagged synapses, and negative values weaken them.

In summary, the proposed model combines three components: NEAT acts as the architect that builds the network topology starting from minimal structures; LIF neurons are the processing engine, which uses binary spikes and can reduce the number of costly operations; and R-STDP is the teacher, a reward-driven local rule that adapts the synaptic weights during play according to behavioural success and not only spike timing.

Chapter 3

State of the Art

This chapter positions the proposed Spiking-NEAT controller with R-STDP against existing work. Because the project draws on several research communities, the review is organised around four questions: how game-playing controllers are learned (with or without gradients), what computational substrate they use (conventional or spiking networks), whether they use local plasticity, and whether their energy cost is analysed. Sections 3.1 to 3.5 review each area, and Section 3.6 compares the works and states the research gap.

3.1 Game-playing controllers

Arcade games and platformers are standard testbeds for learning control from interaction. The Arcade Learning Environment [16] offers a common interface to Atari 2600 games, and the Mario AI Benchmark [17], built around a Super Mario Bros.-style platformer, was used in a series of game-AI competitions.

Gradient-based deep reinforcement learning. Deep Q-networks reached human-level performance on a large set of Atari games, learning from screen pixels and the game score alone [13]. This approach trains deep networks by backpropagation and is data-hungry: DQN, for example, was trained on 50 million frames per game. The computational cost is paid mostly during training, but the resulting controller is a dense floating-point network that is evaluated at every decision step.

Neuroevolution. Evolutionary methods optimise a population of controllers using only episode-level fitness, so they need neither gradients nor labelled data. Hausknecht et al. [18] compared four neuroevolution algorithms, among them NEAT and HyperNEAT, on 61 Atari games with different state representations. They found that direct encodings such as NEAT worked best with compact state representations, whereas the indirect encoding of Hyper-

NEAT scaled to the raw screen. Later work showed that simple genetic algorithms and evolution strategies can be competitive with gradient-based deep reinforcement learning for networks with millions of parameters [19, 20], and Stanley et al. [21] review the field.

NEAT [1] is the best-known method that evolves network structure together with weights, starting from minimal networks (Chapter 2). Its application to Super Mario World in the MarI/O demonstration [6] motivated this project. MarI/O is an informal demonstration rather than a controlled study: it is not peer reviewed and reports no baseline or energy analysis. It nevertheless shows that topology evolution can discover level-clearing behaviour from a compact tile-based observation, without demonstrations or labelled data.

To summarize, the controllers above are conventional continuous-valued networks evaluated densely at every step; performance is reported as score or distance and, to the best of our knowledge, none of these works reports inference energy.

3.2 Spiking networks for control

Gradient-based training. Spikes are not differentiable, but spiking networks can still be trained with backpropagation-style methods through surrogate gradients [14], and recurrent spiking networks can be trained with local approximations of backpropagation through time such as e-prop [22]. In control, these ideas have been combined with deep reinforcement learning. PopSAN trains a population-coded spiking actor together with a deep critic and was deployed on Intel's Loihi chip; on continuous-control tasks it matched deep actor networks trained with the same algorithms, and the authors report roughly 140 times less energy per inference than a deep actor on a Jetson TX2 for HalfCheetah [23]. SANSAC replaces the actor of Soft Actor-Critic with a spiking network, keeps standard critics, and reports near-equivalent performance to the conventional actor on a continuous-control benchmark, all on standard hardware and without an energy measurement [24]. In both cases the spiking network is the actor of an actor-critic scheme, trained with surrogate gradients inside gradient-based reinforcement learning.

Evolutionary training. Qiu et al. [3] use NEAT to evolve a recurrent spiking controller, built from Izhikevich neurons, for nonlinear control problems in continuous domains. They propose two mechanisms for decoding continuous control signals from spikes and report that the spiking controller finds functional solutions significantly faster than sigmoidal networks. Their emphasis is on evolving structure and decoding spikes, not on learning during an episode. Pan et al. [25] evolve brain-inspired modular architectures for spiking networks with a multi-objective algorithm that optimises validation error and total spike count (an energy proxy), but they evaluate on image-classification benchmarks, both static and neuromorphic,

rather than on sequential control.

3.3 Reward-modulated plasticity in spiking agents

Reward-modulated STDP was proposed to solve the distal-reward problem and to implement reinforcement learning in spiking networks [8, 9, 10], and Frémaux and Gerstner [11] place it in the general framework of three-factor learning rules (Section 2.3.2). Applications fall into two groups.

In vision, Mozafari et al. [4] train a deep convolutional spiking network with R-STDP in its higher layers, using first-spike latency coding, and recognise MNIST digits without an external classifier; the reward signal depends on whether the network's decision was correct. In control, Bing et al. [26] use R-STDP synapses to learn lane keeping end to end on a Pioneer robot, with input from a neuromorphic vision sensor, and Yang et al. [27] derive an R-STDP method from the global policy gradient and apply it to vision-based navigation in ViZDoom and to robot-control tasks. These works show that R-STDP can learn closed-loop control, so the open question is not whether it can control an agent but under which conditions.

Two limitations recur. First, the network structure is chosen by the designer and fixed before learning. Second, as Yang et al. note, local rules often do not align with the global learning objective, which limits performance [27]. Evolving the structure while using R-STDP only for within-episode adaptation, as proposed here, is a different division of labour between the global and the local learning mechanism.

3.4 Evolved plastic networks

Evolving the learning process, and not only the weights, has a long history. Early work in evolutionary robotics evolved the mechanisms by which a controller self-organises, instead of evolving fixed parameters directly [28]. Soltoggio et al. evolved modulatory topologies, that is, networks that decide where neuromodulatory signals are routed, and showed that they outperform fixed-weight and Hebbian networks in a simulated-bee, reinforcement-learning-like task [29]. Later work studied the evolutionary advantages of neuromodulated plasticity in dynamic, reward-based scenarios [30], and the field has been surveyed under the name Evolved Plastic Artificial Neural Networks (EPANNs) [31].

A few works bring this idea to spiking networks. Lu et al. [32] use Cartesian genetic programming to evolve the mathematical expressions of plasticity rules for LIF networks and report rules that match or exceed R-STDP baselines on XOR and cart-pole; the network structure is fixed. Efe [33], in an MSc thesis, applies NEAT to spiking networks and reports that it solves XOR, pole balancing and food chasing but not problems that require adaptation during

the agent's lifetime, which are solved after adding dopamine-modulated STDP. Pontes-Filho et al. [34] propose the NAGI framework, in which a modified NEAT evolves spiking controllers of LIF neurons whose weights adapt during the agent's lifetime through STDP-based rules assigned to each neuron, evaluated on foraging, logic-gate emulation and cart-pole tasks.

These are the works closest to this project. Their tasks are small (XOR, cart-pole, foraging in simple worlds), their inputs are not pixel-derived, and, to the best of our knowledge, none compares energy against a matched conventional baseline.

3.5 Energy-aware evaluation and neuromorphic hardware

Green AI [5] argues that computational cost should be reported alongside accuracy. At the hardware level, the energy of an operation depends strongly on its type: in the widely cited 45-nm estimates of Horowitz [15], a 32-bit floating-point multiply-accumulate costs several times more than an accumulate, and memory access costs more than arithmetic. Neuromorphic processors such as Loihi [35] implement event-driven spiking computation with on-chip learning; Davies et al. [36] survey results obtained on this platform and Schuman et al. [37] discuss opportunities and open problems for neuromorphic algorithms. Control-theoretic foundations for neuromorphic controllers are still being developed [38].

Energy claims for spiking networks are of two kinds: measurements on neuromorphic hardware, as in the PopSAN work above, and operation-count models evaluated on conventional hardware. The second kind is easy to apply, and it is the approach adopted in this project (Section 2.2.4), but it ignores memory and communication costs. Its results should therefore be read as comparative estimates between two models under explicit assumptions, not as predictions of power consumption on a given device.

3.6 Comparative analysis and research gap

Table 3.1 compares the works reviewed along the dimensions introduced at the start of the chapter.

Three patterns emerge from the table. First, gradient-free game-playing agents use conventional networks and do not analyse inference energy. Second, spiking controllers are either trained offline with global, gradient-based methods, or evolved without lifetime adaptation, whereas R-STDP controllers adapt through local rules but with a structure fixed by the designer. Third, the few works that combine evolution, spiking dynamics and plasticity are evaluated on small tasks, without pixel-derived input and without an energy comparison against a matched conventional baseline.

To the best of our knowledge, no existing work combines (i) NEAT-style structural evo-

lution, (ii) LIF dynamics and (iii) R-STDP adaptation within an episode in a sequential platform game with pixel-derived input, and compares it with a matched NEAT baseline using an explicit inference-energy model. This project addresses that gap through the following questions, which the experiments of this work will answer:

1. Can a topology-evolved spiking controller with R-STDP reach at least 90% of the baseline performance, with no more than 10% additional completion time, on Level 1-1?
2. How does its estimated inference energy compare with that of the NEAT baseline under the operation-count model of Section 2.2.4?
3. How much does R-STDP contribute beyond evolution alone? This is examined through ablations (R-STDP disabled; Lamarckian write-back).

Table 3.1: Positioning of related work. "n/r": not reported in the cited work. "Local plasticity" means that weights change through local (STDP-type) rules; it does not indicate whether learning continues after deployment.

Work	Network	Learned or evolved	Local plasticity	Task	Energy
Mnih et al. [13]	ANN	Weights (backprop.)	No	Atari games	n/r
Hausknecht et al. [18]	ANN	Weights, topology (neuroevolution)	No	Atari games	n/r
MarI/O, NEAT [1, 6]	ANN	Topology, weights (NEAT)	No	Super Mario World	n/r
Qiu et al. [3]	SNN	Topology, weights (NEAT)	No	Nonlinear control	n/r
Pan et al. [25]	SNN	Architecture (evolutionary search)	No	Image classification	Spike count (proxy)
Tang et al. [23]	SNN actor	Weights (deep RL, surrogate gradients)	No	Continuous control	Measured (Loihi)
Hunter et al. [24]	SNN actor	Weights (SAC, surrogate gradients)	No	Continuous control	n/r
Mozafari et al. [4]	SNN	Weights (STDP, R-STDP)	Yes	Digit recognition (MNIST)	Qualitative
Bing et al. [26]	SNN	Weights (R-STDP)	Yes	Lane keeping (robot)	n/r
Yang et al. [27]	SNN	Weights (R-STDP)	Yes	ViZDoom, robot control	n/r
Soltoggio et al. [29, 30]	ANN	Modulatory topology, plasticity	Yes	Simulated foraging, dynamic reward tasks	n/r
Lu et al. [32]	SNN (LIF)	Plasticity rules (genetic programming)	Yes	XOR, cart-pole	n/r
Efe [33]	SNN	Topology (NEAT), dopamine-modulated STDP	Yes	XOR, pole balancing, foraging	n/r
Pontes-Filho et al. [34]	SNN	Modified NEAT, STDP-based rules	Yes	Foraging, logic gates, cart-pole	n/r
This work	SNN (LIF)	NEAT topology and initial weights; R-STDP within episodes	Yes	Super Mario Bros (pixel-derived input)	Estimated

Bibliography

[1] K. O. Stanley and R. Miikkulainen, "Evolving neural networks through augmenting topologies," *Evolutionary Computation*, vol. 10, no. 2, pp. 99–127, 2002.

[2] W. Gerstner, W. M. Kistler, R. Naud, and L. Paninski, *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition*. Cambridge University Press, 2014.

[3] H. Qiu, M. Garratt, D. Howard, and S. Anavatti, "Evolving spiking neural networks for nonlinear control problems," in *2018 IEEE Symposium Series on Computational Intelligence (SSCI)*. IEEE, 2018, pp. 1367–1373.

[4] M. Mozafari, S. R. Kheradpisheh, T. Masquelier, A. Nowzari-Dalini, and M. Ganjtabesh, "First-spike-based visual categorization using reward-modulated STDP," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 29, no. 12, pp. 6178–6190, 2018.

[5] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2020.

[6] SethBling, "MarI/O - machine learning for video games," https://www.youtube.com/watch?v=qv6UVOQ0F44, 2015, accessed: 2026-09-08.

[7] D. O. Hebb, *The Organization of Behavior: A Neuropsychological Theory*. Wiley, 1949.

[8] E. M. Izhikevich, "Solving the distal reward problem through linkage of stdp and dopamine signaling in the brain," *Cerebral Cortex*, vol. 17, no. 10, pp. 2443–2452, 2007.

[9] R. V. Florian, "Reinforcement learning through modulation of spike-timing-dependent synaptic plasticity," *Neural Computation*, vol. 19, no. 6, pp. 1468–1502, 2007.

[10] R. Legenstein, D. Pecevski, and W. Maass, "A learning theory for reward-modulated spike-timing-dependent plasticity with application to biofeedback," *PLoS Computational Biology*, vol. 4, no. 10, p. e1000180, 2008.

[11] N. Frémaux and W. Gerstner, "Neuromodulated spike-timing-dependent plasticity, and theory of three-factor learning rules," *Frontiers in Neural Circuits*, vol. 9, p. 85, 2016.

[12] C. Kauten, "Super mario bros for openai gym," https://github.com/Kautenja/gym-super-mario-bros, 2018, gitHub repository.

[13] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[14] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.

[15] M. Horowitz, "1.1 computing's energy problem (and what we can do about it)," in *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers*. IEEE, 2014, pp. 10–14.

[16] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, "The Arcade Learning Environment: An evaluation platform for general agents," *Journal of Artificial Intelligence Research*, vol. 47, pp. 253–279, 2013.

[17] S. Karakovskiy and J. Togelius, "The Mario AI benchmark and competitions," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 55–67, 2012.

[18] M. Hausknecht, J. Lehman, R. Miikkulainen, and P. Stone, "A neuroevolution approach to general Atari game playing," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 6, no. 4, pp. 355–366, 2014.

[19] F. P. Such, V. Madhavan, E. Conti, J. Lehman, K. O. Stanley, and J. Clune, "Deep neuroevolution: Genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning," arXiv preprint arXiv:1712.06567, 2017.

[20] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever, "Evolution strategies as a scalable alternative to reinforcement learning," arXiv preprint arXiv:1703.03864, 2017.

[21] K. O. Stanley, J. Clune, J. Lehman, and R. Miikkulainen, "Designing neural networks through neuroevolution," *Nature Machine Intelligence*, vol. 1, no. 1, pp. 24–35, 2019.

[22] G. Bellec, F. Scherr, A. Subramoney, E. Hajek, D. Salaj, R. Legenstein, and W. Maass, "A solution to the learning dilemma for recurrent networks of spiking neurons," *Nature Communications*, vol. 11, p. 3625, 2020.

[23] G. Tang, N. Kumar, R. Yoo, and K. P. Michmizos, "Deep reinforcement learning with population-coded spiking neural network for continuous control," in *Proceedings of the 2020 Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 155, 2021.

[24] J. Hunter, M. M. H. Shuvo, and K. Roy, "Spiking neural networks for continuous control: Neuromorphic reinforcement learning in conventional computing," *arXiv preprint arXiv:2608.22729*, 2026, iCLR 2026 Second Workshop on World Models.

[25] W. Pan, F. Zhao, Z. Zhao, and Y. Zeng, "Brain-inspired evolutionary architectures for spiking neural networks," *arXiv preprint arXiv:2309.05263*, 2023.

[26] Z. Bing, C. Meschede, K. Huang, G. Chen, F. Röhrbein, M. Akl, and A. Knoll, "End to end learning of spiking neural network based on R-STDP for a lane keeping vehicle," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 4725–4732.

[27] Z. Yang, S. Guo, Y. Fang, Z. Yu, and J. K. Liu, "Spiking variational policy gradient for brain inspired reinforcement learning," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 47, no. 3, pp. 1975–1990, 2025.

[28] D. Floreano and J. Urzelai, "Evolutionary robots with on-line self-organization and behavioral fitness," *Neural Networks*, vol. 13, no. 4–5, pp. 431–443, 2000.

[29] A. Soltoggio, P. Dürr, C. Mattiussi, and D. Floreano, "Evolving neuromodulatory topologies for reinforcement learning-like problems," in *2007 IEEE Congress on Evolutionary Computation*, 2007.

[30] A. Soltoggio, J. A. Bullinaria, C. Mattiussi, P. Durr, and D. Floreano, "Evolutionary advantages of neuromodulated plasticity in dynamic, reward-based scenarios," *Proceedings of the 11th International Conference on Artificial Life*, pp. 569–576, 2008.

[31] A. Soltoggio, K. O. Stanley, and S. Risi, "Born to learn: the inspiration, progress, and future of evolved plastic artificial neural networks," *Neural Networks*, vol. 108, pp. 48–67, 2018.

[32] J. Lu, J. J. Hagenaars, and G. C. H. E. de Croon, "Evolving-to-learn reinforcement learning tasks with spiking neural networks," arXiv preprint arXiv:2202.12322, 2022.

[33] A. Efe, "Evolutionary adaptation and dopamine modulated learning in spiking neural networks," Master's thesis, Bilkent University, 2021.

[34] S. Pontes-Filho, K. Olsen, A. Yazidi, M. A. Riegler, P. Halvorsen, and S. Nichele, "Towards the neuroevolution of low-level artificial general intelligence," *Frontiers in Robotics and AI*, vol. 9, p. 1007547, 2022.

[35] M. Davies, N. Srinivasa, T.-H. Lin, G. Chinya, Y. Cao, S. H. Choday *et al.*, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.

[36] M. Davies, A. Wild, G. Orchard, Y. Sandamirskaya, G. A. F. Guerra, P. Joshi, P. Plank, and S. R. Risbud, "Advancing neuromorphic computing with Loihi: A survey of results and outlook," *Proceedings of the IEEE*, vol. 109, no. 5, pp. 911–934, 2021.

[37] C. D. Schuman, S. R. Kulkarni, M. Parsa, J. P. Mitchell, P. Date, and B. Kay, "Opportunities for neuromorphic computing algorithms and applications," *Nature Computational Science*, vol. 2, no. 1, pp. 10–19, 2022.

[38] T. Medvedeva, A. Franci, and F. Castaños, "Formalizing neuromorphic control systems: A general proposal and a rhythmic case study," arXiv preprint arXiv:2506.10203, 2025, submitted to the 64th IEEE Conference on Decision and Control.